

AI ASSISTED PROGRAMMING

MICHAEL FEATHERS

AI Assisted Programming

Michael Feathers

This book is available at
<https://leanpub.com/ai-assisted-programming>

This version was published on 2025-05-26



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2025 Michael Feathers

Contents

Introduction	i
Understanding Assistants Through Their Behavior	1
Surfacing	4
Attention	8
Dissipation	10
Pattern Preference	10
Concepts	12
Eagerness	13
Batch Mentality	14
Clamping	15
Roughouts	16
Ownership	18
Techniques	20
Make Projections	21
Use Waywords	27
Notice Names	36
Generate From Tests	37
Test-Driven Development	38
Ask For N	39
Pidgin Specification	41
Step Check	47
Understand the Generated	49
Generate Reference Model	50

Go Rogue	51
Reduce Step Size	52
Ask What's Unnecessary	53
Ask For Reflection	54
Vary Resolution	55
Stabilize Iteratively	56
Generate And Deploy	57
Completion Check	58
Solve Then Ask	59
Ask For Review	60
Let It Drive	61
Ask For Summary Prompt	62
Generate Tests	63
Ask For Standards Check	64
Ask Again	65
Approach Obliquely	66
Interrupt Flow	67
Resources	68

Introduction

LLM-based technologies are being used nearly everywhere. In software development they are impacting the way that we create, understand and modify code.

In some ways, they are “nothing new” for a developer. We have had code completion, code generation, and code analysis tools for decades. However, things are different now. We can generate scaffolding quickly and run elaborate code experiments at nearly zero cost.

This is great but it isn’t all upside. Now, it is easier to create code but verification is harder. We have to understand and compare solutions almost continuously. Oddly, this enhances our creativity and increases our toil at the same time. The human side of the equation — how we manage our energy and mental clarity as we work — are challenges that will require more attention.

It might seem strange to write this book now. Many things are changing however I have confidence that some principles and practices will be stable over time.

Understanding Assistants Through Their Behavior

“All models are wrong, some are useful.”

— George Box

Whenever we try out a new technology there are several ways to approach it. If we are technically-inclined we might learn its internals and reason from them to form our understanding. If we just want to approach it as a user we might explore its behavior and use it to help us develop our intuition.

As I write this, most AI assistants use Generative Pre-trained Transformer (GPT) technology in LLMs (Large Language Models). the technology is rather complicated. If you want to learn about it you can read papers, watch videos and attempt to form an inside-out understanding, but there is only so far that many of us can go. The technology rests on a basis of linear algebra and recurrent neural network fundamentals that make it hard to reason about it “from the ground up.”

In this section of the book, I am going to use the opposite approach. I’m going to present an experiential model of LLMs — things that you may notice when you work with them — and then move inward to the mechanisms that produce those behaviors. We’ll end up with a simplified model that helps us understand what is easy, what is hard, and why.

I'll introduce the model by describing several phenomena that are apparent from use:

- Surfacing
- Attention
- Dissipation
- Pattern Preference

All of these are intrinsic to GPT technology. Assistants often use additional technologies to deal sidestep the limitations of some of these characteristics, but I have the sense that all of them will continue to be noticable.

When you understand them you can use AI more effectively.

Surfacing

Imagine that you have a space in front of you with many points. Each of these points represents a concept.

Now imagine that there is a line between every pair of related concepts.

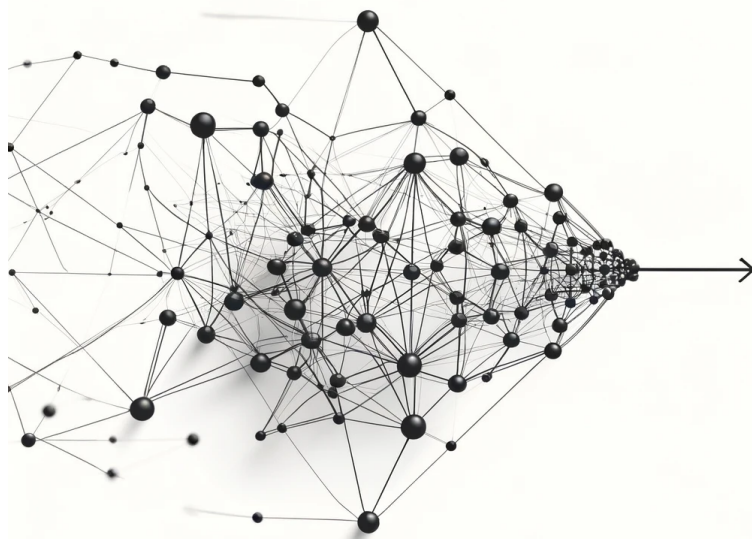


The current generation of LLMs are trained on very large text corpuses. We can expect that every concept is related to every other concept in some manner, even if only in a very loose way.

Now, imagine prompting for something. Prompts consist of groups of words that refer to concepts. When your prompt refers to one or more concepts, those concepts are pulled toward you.



When concepts move toward you, closely-related concepts are pulled along also. It is as if they are connected to each other by strings.



When concepts are closer to you they are in the current context — they are more likely to show up in responses. Any concepts that you co-create with the LLM become connected to them.

This is how we generate knowledge when we work with LLMs.



Attention

(How the concept of attention is central to GPT technology and how it manifests in their behavior. Also, the ways that it mirrors human attention has many of the same issues/problems)

Noticing Attention

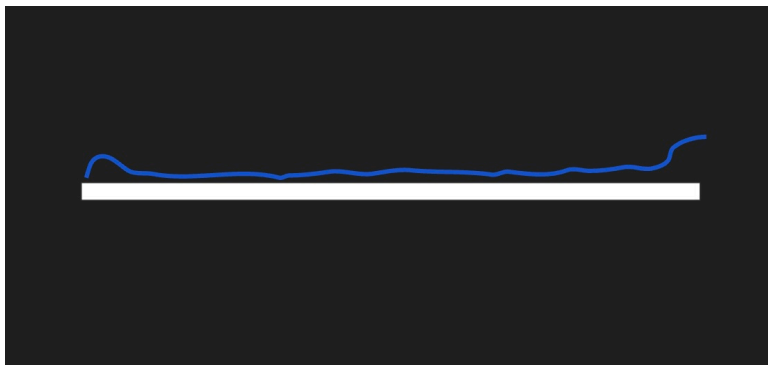
(How it manifests in behavior..)

Understanding the Context Window

(A conceptual model..)

Lost in the Middle

LLMs are prone to something called the *Lost in the Middle Problem*.



Studies have shown that the information that introduced first in a session and the information that is has been introduced most recently both have higher priority than anything in the middle.¹

I'm not surprised.

Human memory has the same characteristics.

It is called the *Serial Position Effect*.²

There are a number of theories about why this happens in humans. I don't know what the answer is but I think that it is interesting to note that both human minds and LLMs differentiate between short and long term memory and both have the characteristic of making memories more available when they are frequently used.

(Elaborate this section with more detail..)

¹Liu, et al. Lost in the Middle: How Language Models Use Long Contexts
<https://arxiv.org/abs/2307.03172>

²The Serial Position Effect https://en.wikipedia.org/wiki/Serial-position_effect

Dissipation

In the *Attention* section, I described *knowledge loss*. It is an example of *dissipation*.

Remember that when we mention a concept or work with one in our prompts, it is pulled closer to us. However, over time it tends to move back to where it was. It becomes less likely to “come up” in our interactions. This is very much like trying to remember something and discovering that it is just beyond your grasp. If you are able to remember it, it is pulled into the foreground again. If you continue to access it, it stays there. When you stop accessing it, it starts to recede again.

You might wonder why I call this *dissipation*. I call it that because often we notice it through a fragmentation of knowledge. We explain a concept and then after many other interactions, we discover that the LLM’s understanding fades. It gives you more “almost, but not quite right” responses. They are working on portions of a concept rather than the complete idea. It is a bit like human memory. We might remember part of something but not all of it.

Remember that when we mention a concept or work with one in our prompts, it is pulled closer to us, but there is only so much space in the context — we can’t have too many things there at once. Over time, things in the context become less available. They shift back. It very much like remembering something that happened a long time ago. If a memory is old, you are less likely to think about it everyday. But, if you start to think about it frequently it becomes more available to you.

Dissipation is not just an LLM phenomenon. It happens to people all of the time.

Pattern Preference

(How base LLM technology uses Daniel Kahneman's's System 1 by default and how the more recent reasoning models move it to System 2)

Concepts

Eagerness

As I write this, LLM-based assistants are in common use. We use them for knowledge queries, assistance while writing documents and general problem solving.

There is tension between the generic assistants that are offered to the public at large and the ones that are specialized for specific domains or interaction styles. The generic ones have an interesting quirk that impacts us. I call this quirk *eagerness*. You've probably experienced it.

A tool is *eager* when it always attempts to fully satisfy your request. For generic tools that are meant to do anything it makes sense for them to be *eager*. Often these tools are in competition with each other and, let's face it, users don't like to be told "no." That's fine when correctness doesn't matter much, but sometimes we (developers) should be told no, particularly when there isn't enough information to satisfy a request with reasonable correctness.

In an early experiment, I asked an IDE-based tool to generate a class that would satisfy a set of tests and it told me that it couldn't because it didn't have enough context. I was actually glad it did that.

Some tools let you override this behavior with a prompt like: "I know you don't have enough information but please make your best attempt", or more directly, "Do it anyway." Others don't.

When I encounter a tool with low *eagerness*, I make note of it and I use it when correctness is critical. If it is *eager* and it doesn't allow me to override, I use for brainstorming and ideation - tasks where I'll have to do more research to validate ideas.

Batch Mentality

The hardest thing about learning is unlearning.

(Experienced developers might be the least suited for this tech. We think in terms of determinism. Thought experiment: cloning a person and placing them 3 feet away. The state vector. Immediate differentiation. The differentiation is built-in with AI because of randomness. You can't teach AI something, put it in a box and give it to someone else. If you must think in terms of deliverables, you are creating conversation seeds)

Clamping

(When LLMs are heavily specialized in narrow domains and then less effective overall)

Roughouts

One of the nice things about technical writing is that you are forced to find new ways of explaining things. Sometimes you arrive at a concept that makes something easier to understand but you don't have a word for it. If you are lucky, you might find a word from another domain that is close enough to be useful.

In sculpture and wood carving there's something called a "rough-out." It's the thing that you are working on in its unfinished state. Imagine a sculptor creating a statue of a person. It takes a while for the stone to start to look like a human. The working state of the stone is the "roughout."

In AI-assisted programming you're always working on something. It can be a method, a class, a service or just a concept. In all of these cases, the thing you are working on is not the finished work. At the very least, it requires review. Often you will carry that item (the roughout) through many iterations before you are comfortable with it. Many times you will just discard it if you've reached a dead end in your exploratory path.

I use the word "roughout" to describe whatever I am currently working on with AI. Often I'll say "rough" for short.

Why is it good to have a word for this?

There are two reasons:

There are many different types of things that we work on with AI assistance. Regardless of their type it useful to have a generic name for them. It makes communication easier.

The second reason relates to the first.

The current generation of tools have a knowledge management problem. In the Context Window section I describe the performance degradation that tends to happen in long-lived (overused?) chats/sessions. At a certain point, you just have to start a new

session. The problem is that you don't want to start fresh. You want the rough and whatever knowledge and "know how" you have generated. You can manually copy all of the prompts and responses from the original session to a new one and hope that is sufficient but you might end up with the same problem.

I know that there are people working on the knowledge management problem. We'll likely have better solutions soon. At the very least, we can identify the roughs we are working on and see them as distinct from our sessions. If we are working on them they certainly are *things*.

Ownership

Humans have a tricky relationship with technology. We want our tools to do things for us but they aren't as capable as we are. They might be better at some tasks but they never have the same context as a human. A big part of this context is responsibility. The world looks different when you are responsible.

Over the past decade or so as AI has become more capable, many people have made the argument that "it" is responsible for various things. This idea predates contemporary AI. I've seen companies argue that "it wasn't us, it was the algorithm." Anyone who knows a thing or two about the legal system knows that humans hold humans accountable. That's where the buck stops.

I've framed this as a rather large thing but it also happens at micro-scale. When automated refactoring tools were new, the people who developed them held themselves to a very high standard. Their goal was to make each use of the refactoring feature safe and disallow any attempt that would not preserve behavior. When we have that assurance, we have far fewer things to worry about – there's a whole space in our workflow where we know that there is no way that we could be introducing hazards. This standard is extreme. It is almost a test. If your tool can do the analysis for a complicated refactoring and only allow it when it preserves behavior, it meets the goal.

Imagine something different now.

You use a refactoring tool and ask it to do a non-trivial refactoring for you – it does most of the work but leaves the code in a (slight) uncompileable state.

That has happened to me a few times and I've often wondered whether it was intentional. I don't think it really matters either way. The interesting thing is how we react in that situation – we realize that we are in charge. We have to take ownership of the

work. This is always true, but it is nice to have a tool give us that reminder. We can't abdicate responsibility.

Whenever I see a tool acting this way, I think of it as an ownership maneuver. In today's chat interfaces I see the same behavior. You paste in some code and ask it how to make a change. Rather than giving you code that you can paste back, sometimes it gives you a summary or something slightly incomplete.

As I mentioned earlier, I don't know whether this is intentional or not. I just know that I don't see it as a bug. I see it as a feature.

Techniques

Make Projections

When we change our perspective we notice different things. We can change our perspective by stepping from our work or we can do something more radical.

Here is something to try.

Take some piece of code that you are working on and ask your assistant to turn it into math. This may not be very useful if your code is just string manipulation or API-intensive glue code. In other cases you might be surprised by this projection of the *rough*.

Here is the code for an Updater class.

```
1  class Updater
2
3      def initialize io
4          @io = io
5      end
6
7      def run
8          actions = @io.read_actions.lines
9          updates = @io.read_updates.lines
10
11          due_updates = due(updates)
12          non_due_updates = non_due(updates)
13
14          new_actions = due_updates + actions
15          @io.write_actions(new_actions)
16
17          sorted_non_due = non_due_updates.sort_by do |line|
18              Day.from_text(line.split.first).date
19          end
20          @io.write_updates(sorted_non_due)
21      end
```

```
22
23   def due us
24     us.select {|e| due?(e)}
25     .map {|e| strip_date(e)}
26   end
27
28   def non_due us
29     us.reject {|e| due?(e)}
30   end
31
32   def strip_date line
33     line.split.drop(1).join(" ") + $/
34   end
35
36   def due? line
37     Day.from_text(line.split.first).date <= @io.today.date
38   rescue
39     false
40   end
41
42 end
```

Unfortunately, the assistant gave me the math in natural language. I decided to ask for it in LaTeX, a language for typesetting papers with mathematics. Then I took it and pasted into an online tool that renders LaTeX. This is what it gave me:

Let:

A = set of actions

U = set of updates

$D = \{u \in U \mid \text{date}(u) \leq \text{today}\}$ (due updates)

$N = U \setminus D$ (non-due updates)

Operations:

$A' = D \cup A$ (new actions)

$N' = \text{sort}(N)$ by date

Output:

Write A' to actions file

Write N' to updates file

Where:

$\text{date}(u)$ extracts the date from update u

$\text{sort}(N)$ arranges elements of N in ascending order of date

This mathematical view is much clearer.

We still have the hallucination problem. There is no guarantee that this math is correct, but that doesn't make it less valuable. Projections give us vantage points that we can use to think about the *rough* differently. If the projection does not match our understanding of the *rough*, it could be a problem in our understanding or a problem with the projection. Either way, we have some investigation to do and we will learn a lot along the way.

When you want to make a projection, think about any possible representation that could give you a different point of view. You might want to stop short of asking for your code as a Shakespearean sonnet, but asking for a projection of your code into another programming language can be very enlightening.

If the code I am working on is in *Java*, I might ask for a *Go* projection. If the code is highly procedural, I might ask for one

in *Haskell* just to see functional coding opportunities.

One of my favorite projections is the *state machine* view. Complex systems often have hidden states. The state machine view can expose them. Here I queried for a projection of a simple class as a state machine in PlantUML syntax:

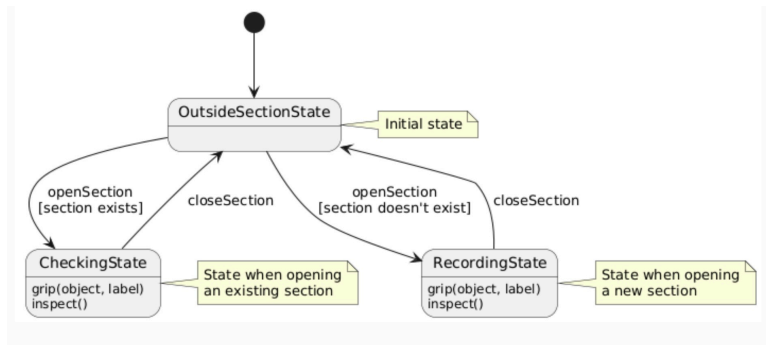
```
1  @startuml
2  [*] --> OutsideSectionState
3
4  state OutsideSectionState
5  state CheckingState
6  state RecordingState
7
8  OutsideSectionState --> CheckingState : openSection\n[sec\
   tion exists]
9  OutsideSectionState --> RecordingState : openSection\n[se\
   ction doesn't exist]
10
11 CheckingState --> OutsideSectionState : closeSection
12 RecordingState --> OutsideSectionState : closeSection
13
14 CheckingState : grip(object, label)
15 CheckingState : inspect()
16
17 RecordingState : grip(object, label)
18 RecordingState : inspect()
19
20 note right of OutsideSectionState
21   Initial state
22 end note
23
24 note right of CheckingState
25   State when opening
26   an existing section
27 end note
```

```

28
29 note right of RecordingState
30     State when opening
31     a new section
32 end note
33 @enduml

```

I copied the response into a *PlantUML* renderer and got the diagram I wanted:



I like to see projections as different views of the same thing from different angles. We'd like each of them to describe the same underlying thing in the same way that static and dynamic diagrams present different views of a system in modeling. We'd like these views to be consistent with each other but in reality they may be slightly off or lossy. That doesn't make them less useful.



Imagine seeing a garden from only one viewpoint. There may be many things that you miss. Walking around, and seeing the garden from many different points of view gives you a more complete picture. This is vitally important if you are the gardener.

In many cases, your projections are text. Seeing *C* code as

equivalent *Rust* is a good example.

Diagrammatic projections can be challenging. If you are lucky, your assistant has plugins that can give you good visualizations. However, it is important to realize that text representations wonderful because you can usually feed them into existing tools to get exactly what you want.

The tension between generic and specialized tools is acute here. I often use the most capable LLM I can find for intense work. I don't mind copying text into other tools. It seems to be worth the effort compared to using a lesser LLM just because it does a good job rendering diagrams.

Use Waywords

One of the most interesting aspects of LLM technology is its ability to infer context. If you use the word *property* in a sentence that contains legal terminology, it will correctly infer that you mean “legal property.” If the sentence contains design terminology the inferred meaning of *property* might be “an aspect of a thing.”

This is good but it is a bit disorienting if you are used to precise communication. We have all been in technical conversations where people talk past each other because they use the same words to mean slightly different things. In many cases, these meanings can be glossed over. In other cases, they are absolutely critical.

Programming is precise communication.

System behavior gives our code unambiguous meaning. We can attempt to get that same level of precision by inventing words and using them to refer to concepts and things in our sessions.

A Wayword for a Refactoring

A while ago, I was doing some refactoring. As an intermediate step, I needed to change the use of one type to another within a method. I wanted to do this while keeping the method signature the same.

After a few attempts I ended up with this prompt:

“Convert all uses of `Date` to `DateTime` in this method but maintain the public interface. This will require converting all `Date` parameters to `DateTime` after they have been passed into the method and converting the return value to `Date` if necessary. If there are `Date` instance variables, do not convert them. Instead, convert `DateTime` to `Date` or `Date` to `DateTime` as needed to interact with them.”

This prompt worked well. I had written it in a general way, hoping that it would work for any method.

Before I applied it to another method I had a decision to make. I could just prompt directly:

“Do this to the *X* method also.”

Or, I could find an easier way to refer to this new operation.

I decided that I would call it *intralate* (internal translate). I wanted to be able to say:

“Intralate type *X* to type *Y* in method *M*.”

Intralate is a completely made up word – a *neologism*. In the context of prompting I like to call these neologisms *waywords*. They are waypoints in a conversation and, sometimes, words that describe ways of doing something.

“Call the operation I just performed ‘intralate’. I want to be able to say: Intralate type *X* to type *Y* in method *M*.”

Again, it worked well.

As an exercise, I tried to generalize *intralate* further. I wanted to be able to *intralate* a type in a method but also in a class, or a service.

In all of these cases, you need a notion of the *interface* that is being preserved. When you intralate methods, instance variables and method calls within the method are part of the interface – they interface with the rest of the class (and the rest of the world). In

contrast, when you *intralate* a class, they are part of the “inside” – the types of instance variables and the types in internal method calls should always be converted.

It would be easy to make a multi-clause prompt to handle both of these cases, but I wanted to define *intralate* concisely, I needed a way to refer to the “inside” and the “outside” of a generic abstraction. Sadly, we don’t have precise words for those concepts.¹

Regardless, the fact that we can form these concepts is important. There are many things that we can conceptualize that we don’t have words for. Sometimes they are the most interesting things.

It might seem extreme to make up a word to describe an operation but it can be a good choice when there is no word that fits. The advantage is that we can completely disambiguate ourselves from all of the connotations of existing words. This can be very useful. Think of it as introducing a named variable or method in a session. The name makes reference and use easier.

A Simpler Wayword

Let’s look at another *wayword* in more detail.

I like to use method chains. Code written with them can be easier to read. I asked an assistant to refactor some existing code toward this chained style and it gave me this:

¹*Scope* is the closest word I’ve found but it is rather ambiguous.

```
1  def load_command_log
2      @log ||= @list.io
3          .read_log
4          .split($/)
5          .map { |line| line.split(',') }
6          .select { |items| items.size == 2 }
7          .map { |k, v| [k, v.to_i] }
8          .to_h
9          .tap { |h| h.default = 0 }
10 end
```

It was nice, but I didn't like the use of the `tap` function. `Tap` returns the `HashMap` that it is called upon, but only after running a block that can modify it. In this case, the block sets the `default` property of the map to zero.

While the code isn't hard to understand, I really prefer to have side-effect free chains when I can.

I gave the assistant the following prompt:

“Change this code so that every statement is either side-effect free or not:”

Here is the response:²

²I've shortened some variable names to make the generated code fit within page margins.

```
1  def load_command_log
2    # Non-side-effect free operation (memoization)
3    @log ||= begin
4      # Side-effect free operations
5      lines = @list.io.read_log.split($/)
6      valid_items = lines.map { |line| line.split(',') }
7                          .select { |items| items.size == 2 }
8      parsed_items = valid_items.map { |k, v| [k, v.to_i] }
9      hash = parsed_items.to_h
10
11     # Non-side-effect free operation (modifying the hash)
12     hash.default = 0
13
14     hash
15   end
16 end
```

The code satisfied my tests but it wasn't quite what I was looking for. To get a better idea of what it understood from my prompt, I asked for a summary.

“Explain the effect of my last prompt. Be brief.”

Your last prompt asked me to modify a Ruby code snippet. Specifically, you requested that I change the code so that every statement is either side-effect free or not. This involved separating operations that modify program state from those that don't, and restructuring the code accordingly.

After trying a few more things out and thinking about what I really wanted I typed this:

“Change this code so that it meets the following criteria: any method chains that contain side effects are put in different statements. All contiguous sequences of statements with method chains that have no side effects are merged into a single statement (or return expression). Do not introduce blocks and do not reuse local variables.”

It replied with this code:

```
1 def load_command_log
2   @log ||= @list.io
3       .read_log
4       .split($/)
5       .map { |line| line.split(',') }
6       .select { |items| items.size == 2 }
7       .map { |k, v| [k, v.to_i] }.to_h
8   @log.default = 0
9   @log
10 end
```

This is close enough to what I wanted, and it passes my tests.

Now, let's turn it into a *wayword*:

“I want to do this operation on many methods. Call this operation ‘floya.’ Perform it whenever I ask for the ‘floya’ of a method.”

Here is the response:

Certainly, I understand that you want to define an operation called ‘floya’ that can be applied to Ruby methods. The ‘floya’ operation involves:

Merging contiguous sequences of statements with method chains that have no side effects into a single statement (or return expression). Separating any method chains that contain side effects into different statements. Not introducing new blocks. Not reusing local variables.

I’ll keep this in mind, and whenever you ask for the ‘floya’ of a method, I’ll apply these transformations. Just to confirm, here’s a summary of what ‘floya’ does:

*Combines chains of pure operations into one line
Separates operations with side effects Preserves overall
functionality Doesn’t introduce new blocks or reuse
local variables*

I made up the word *floya* on a whim. After thinking about it for a while, I decided that the made up word *chainify* might be better so I asked the assistant to use it as a synonym.

When you have a good *wayword*, you’ll likely want to use it in other sessions. With current technology, a good way to attempt this is to ask for a *summary prompt*:

“Give me a prompt that I can use in other sessions to chainify methods.”

Here's a prompt you can use in other sessions to request the 'chainify' transformation for Ruby methods: "Please apply the 'chainify' transformation to the following Ruby method. The 'chainify' transformation involves:

Merging all contiguous sequences of statements with method chains that have no side effects into a single statement or return expression. Separating any method chains that contain side effects into different statements. Not introducing new blocks. Not reusing local variables. Preserving the overall functionality of the method.

Here's the method to transform: (paste here)

Please provide the 'chainify' transformed version of this method." You can copy and paste this prompt, replacing the placeholder with the actual Ruby method you want to transform. This should allow you to request the 'chainify' transformation consistently across different sessions or with different AI assistants.

This doesn't always work, but quite often it does. It will be great to have a more direct way to transfer *roughs*. Hopefully, we'll have one soon.

Waywords As Labels

The examples I've shown so far have all been rather involved. All of them are about defining an operation and giving it a name so that it can be used later. This is a great thing to attempt, but often my use of waywords is more mundane. If I use an assistant to produce an examples and I think I might want to refer to it later, I just label it.

"Label this code 'nested factory version.'"

That is enough to be able to refer to the example later in the session. Most assistants will allow you to abbreviate this in various ways. In the one I am using now I can just type

“Label this: nested factory”

Try things out. Find a style that works for you.

Notice Names

When you work with an LLM-based assistant you are having a conversation. Invariably, both you and the assistant will use many terms. The terms you use are based on your knowledge of the domain and your history as a human being – everything you’ve ever read or discussed with anyone.

The assistant has a very different context – one that is informed by all the data that it was trained on and any data that it has access to.

Assistants will often use different words to describe things when they generate responses. If you’ve used the technology for a while, you probably aren’t very surprised when this happens.

I think that is a mistake – we *should* allow ourselves to be surprised.

If you ask for a suggestion or some description of something in the *rough*, pay *particular* attention to the names in the response. I often think that they are more important than the rest of the response. They are important because they can trigger different ideas and lead us in new directions even if we don’t fully agree with the assistant’s take.

(Add examples..)

Generate From Tests

Here's a simple experiment that you should try.

Take a small project and delete all of the source code but leave the tests. Ask your assistant to generate all of the code necessary to pass them. The results might surprise you.

Assistants vary in their ability to do this. Some may tell you that the tests aren't informative enough. Others attempt to and succeed to some degree. When they do the code that they produce is often, uh, interesting.

I did this with a small library. It had 11 classes and interfaces, and 3 test case classes. Despite this imbalance, the code was well covered.

I asked the assistant to list all of the tested classes. It correctly inferred that 3 classes were tested directly, and 4 classes and interfaces were implied. I asked it to generate code that satisfied the tests. After dealing with dependency and compilation issues the tests passed but the code was a giant tangle with many variables that mimicked the state implied by the tests. It wasn't great code by any means.

Why should you do this?

For one thing, it gives you some insight into the capabilities of your assistant. It helps you see how gaps in coverage imply gaps in the code. Additionally, I often find more direct implementations. Whether you use them or not, it is good to know that they exist.

As I mentioned, tools vary in their ability to do this well. It isn't a great loss. Specifications (tests) invariably have gaps in behavior. We can use AI to solve that problem too but currently the workflows are an open problem.

Test-Driven Development

When I do strict test-driven development with AI I write tests and generate code after each of them. Invariably, the tools I've used start to falter a bit. It seems like they fixate on the most recent test case even when told to make sure that all of the tests pass. My sense is that a lot of this comes from *dissipation*. In some tools, creating a new session every time I add a test seems to help.

The TDD cycle is:

1. Write a failing test case
2. Write the code that makes it pass
3. Refactor
4. Repeat

When we incrementally generate from tests we are automating step 2. This is significant but we still have to do steps 1 and 3.

We can automate step 1 by generating tests but generating tests and code is *double trouble* – you have two paths with hallucination possibilities. At the very least, you have to review the tests to make sure they specify the intended behavior.

Step 3, refactoring, is a place where AI can help but even if it doesn't, it isn't a big problem. When you take small steps and have tests, refactoring is very easy.

(elaborate this.. the refactoring step and how TDD allows us to isolate problematic generation. The issue of satisfying all tests incrementally.. path dependence)

Ask For N

If you accept the first response an assistant gives you, you could be losing a significant opportunity. We give the first response too much weight. We fail to consider that there could be other, possibly better, answers.

The radical change that this new tooling has enabled is the ability to cheaply explore many paths and alternatives. Not using it is a waste.

The next time you are working with AI try this experiment – repeat your last prompt and see what happens. Often you will get a different answer. We can use the randomness in LLM inference as a feature.

Here is an example in an obscure domain. There is a operation in image analysis called *dilation*. I wanted to see how I could implement it. I gave my assistant the following prompt:

“Show me a binary morphological dilation on a 32x32 matrix in Java.”

The code it gave me used something called the *structuring element approach*.

It was nice to see this approach but I wanted more so I gave the assistant the same prompt again.

“Show me a binary morphological dilation on a 32x32 matrix in Java.”

This time it gave me an implementation using a triply-nested loop.

You can do this as many times as you like. It’s interesting to see how many times an assistant will give you a unique solution.

We can successively ask for the same thing over and over or ask for slight variations and options each time but it is tedious. We can shortcut this process by asking for a specific number of generations up front. I like to use 3 or 4 in most cases.

“Show me three different ways to compute a binary morphological dilation on a 32x32 matrix in Java.”

Asking for 3 or 4 at once moves you into a larger possibility space. I often think of completely different possibilities even if I like one or more of the responses. This is a great strategy, but we have to be aware of the cognitive cost of reviewing several alternatives that may be almost but not quite the same. Work within your tolerance.

Asking for 2 generations is a special case. When we are confronted by two options, we immediately go into comparison mode. We want to see what is the same and what is different. This is an excellent way of exposing detail. You can use this in conjunction with **Lensing** to see how solutions can vary at different scales.

Ask for N is a very general technique. It can be used as an adjunct to nearly every other technique in this book.

(Show code examples..)

Pidgin Specification

Pidgin : a simplified speech used for communication between people with different languages

— Merriam Webster

One of the more disorienting things about LLM-based assistants is the fact that you don't have to be precise. You can misspell words in your prompts and leave out punctuation and they still do a good job of inferring your intent.

Often, you can prompt in a loose style and still get what you want. Interacting with them is completely unlike programming in that way.

How far can this go?

When you want to understand something try this — don't phrase your prompt as a question. Type it as noun-phrase.

The assistant understands it as a question and gives you a response.

Here is an example.

```
1 public class DefaultViseMechanics implements ViseMechanic\
s {
2     private SectionStore store;
3     private MechanicsState state;
4
5     public DefaultViseMechanics(SectionStore store) {
6         this.store = store;
7         state = new OutsideSectionState(store);
8     }
9
10    public void openSection(String sectionName) {
11        state = store.hasSection(sectionName)
```

```
12         ? ((MechanicsState)new CheckingState(store))
13         : ((MechanicsState)new RecordingState(store));
14     state.openSection(sectionName);
15 }
16
17 public boolean isRecording() {
18     return state.isRecording();
19 }
20
21 public void grip(Object object) {
22     state.grip(object, "");
23 }
24
25 public void grip(Object object, String label) {
26     state.grip(object, label);
27 }
28
29 public void closeSection() {
30     state.closeSection();
31     state = new OutsideSectionState(store);
32 }
33
34 public void inspect() {
35     state.inspect();
36 }
37
38 public void release() {
39     store.clear();
40 }
41
42 }
```

After loading this Java class, my AI assistant responds with a summary of the class.

Let's say that I want a deeper understanding of the openSection

method. Rather than typing (or saying) “Explain open section” I just type “openSection” and it gives me an explanation. It might not be the type of explanation that I wanted, but I can elaborate my prompt and ask again.

A lot of this behavior comes from *instruction tuning*, a technique that tool makers have used to train models to be helpful, i.e, follow instructions better. It’s useful to compare this to talking to a co-worker.

If a co-worker of yours knows the `DefaultViseMechanics` class and you just blurt out “open section” their answer is likely to be “Huh?”

Clearly, humans aren’t specifically trained to follow instructions.

Reducing a question to a noun-phrase is a very simple example of *pidgin*, a form of simplified communication that acts as a bridge between different contexts. The contexts of humans and AI tools are wildly different. We can expect pidgin to develop in our work.

An early example of pidgin is a language named *SudoLang*³. Its author created a long prompt telling an assistant to create a pseudo-language for specifying computation. They added the additional requirement that it should be simple and obvious enough that the assistant would *not* need a specification of the language in order to interpret it. This twisty requirement led it to generate a language with functions, variables, constraints and many other interesting features.

I took an example program in *SudoLang* and dropped it into a chat window that had not been primed with the *SudoLang* spec.⁴ It seemed to work well.

³*SudoLang: A Powerful Pseudocode Programming Language for LLMs*, Eric Elliott <https://medium.com/javascript-scene/sudolang-a-powerful-pseudocode-programming-language-for-llms-d64d42aa719b>

⁴This example was inspired by this *SudoLang* example: <https://github.com/paralleldrive/sudolang-llm-support/blob/main/examples/fibonacci.sudo>

Here is the prompt I used (in *SudoLang* syntax)⁵:

```
1 Execute this pseudocode:
2
3 fibonacci = n => {
4     if (n <= 1) return n
5     else return (fibonacci(n - 1) + fibonacci(n - 2))
6 }
7
8 1..20 |> fibonacci |> log
```

Here is the response:

Here's the execution of the pseudocode you provided, which calculates the Fibonacci sequence for the first 20 numbers:

(list of numbers removed for brevity)

Next, I prompted with this:

```
1 Fibonacci program |>
2   transpile(JavaScript):length=very concise
```

Here is the generated code:

```
1 const fib=n=>n<2?n:fib(n-1)+fib(n-2);
2 console.log([...Array(20)].map((_,i)=>fib(i+1)));
```

Re-prompting with “very concise” changed to “concise” gave me the following response:

⁵A few months before I wrote this chapter, the code executed directly without the “Execute this pseudocode:” instruction. When I did it today, the assistant read it as JavaScript and told me that there is no `|>` operator. This is an example of fine-tuning turning into over-tuning, at least for my use case.

```
1  const fibonacci = n => n <= 1 ? n :  
2    fibonacci(n - 1) + fibonacci(n - 2);  
3  
4  const result = Array.from(  
5    { length: 20 },  
6    (_, i) => fibonacci(i + 1)  
7  );  
8  
9  console.log(result);
```

It's worth thinking about what is really happening here. The tool inferred the prompt's meaning based upon all of the code and knowledge of programming that it had been trained on. It “knows” most programming languages and it has “seen” countless examples of their use. It is enough to fill in the gaps in most cases.

SudoLang syntax is unambiguous enough in that context to give a response that has a fair chance of being correct. However, since the tool I used was highly *eager* it probably wouldn't alert me to any potential ambiguities.

Pidgin languages are easy to develop. You'll find yourself doing it over and over again as you simplify your prompting.

Sometimes I use a form of *pidgin* to specify test cases concisely.

In a session where I've loaded the `DefaultViseMechanics` class, I typed:

```
test: grip o -> assert state.grip o
```

The tool generated a *JUnit* test for the `grip` method using a mock for the class's state variable, verifying that the object passed to `grip` makes its way into `state`.

I'm sure that most syntactic variations of that prompt would work well.

In many cases it's good to start with a brief prompt, check the output, and elaborate it only when you need to.

It's important to remember that you are in conversation and that most assistants are tuned to help you. You can just paste in an error message and it will attempt to correct the code it generated in most cases. Often I just type "Try again" if the error is clear.

(Replace the sudolang example with an original one..)

Step Check

Step Checking is the act of asking an assistant for feedback at each step in your workflow.

This technique is one I often forget about.

I decide on a design approach, get some tests passing, and then ask an assistant to review and make suggestions. Quite often, I wish I'd had the feedback sooner.

Late feedback gives us a choice — We can undo our work and start over again or we can ignore the advice and move on to the next thing. Neither is ideal.

Knowing this, it seems reasonable to keep our steps small but we need to pay attention to our prompting also. Small, focused prompts are less likely to derail us but they also give us less feedback. Prompts that ask for deeper and wider analysis can help us learn more about what we have but the cost of acting on them can be higher.

I like to ask the assistant for review as often as I can using focused prompts like “What would be a better name for this class?” When I am about to end an exploration I widen them, putting more of the system under analysis.

At this level, you can have prompts like:

“Suggest design changes that will reduce latency for the following workflows in the application: ... “

Ideally, you will have asked speculative questions like that before doing the work but sometimes questions occur to us late.

By the time I release this book, there might be tools that incorporate broad, AI-based checks into local build pipelines. These checks can join the tests, the linters, the security checks and all of the other steps of a build that give us confidence.

We've always had decisions to make when we solicit feedback. We have to decide what types of feedback we want and when we want them. AI tools make this a more pressing concern.

Understand the Generated

We must understand the code we introduce into our systems.

This is a very stark statement. It is mostly true but there are some exceptions. I'll address those later, but for right now let's assume that understanding is absolutely necessary. We don't have many tools that we can use to make understanding easier. The one that I think is most important is automated testing.

There is a mode that developers often get into that can be very hazardous. We say to ourselves "I'm going to make this change and see if it passes the tests. If it does, I'll assume that it is right."

I don't think I need to explain why this can be a problem. Seasoned developers know that passing tests* are a confidence measure. They augment system understanding but they are not a replacement for it. There's no real substitute. This is why it is very important to understand the things that you generate, especially when they can affect behavior in production.

If you are generating things speculatively, for ideation, taking time to understand them pays off then too. I look at development as a learning exercise that produces code as a side effect. The interesting thing about learning is that it pays off in ways you'd never expect. Everything that you learn is something new that you'll remember when you encounter something similar — another thing that you'll compare and contrast with other things that you learn later.

See the possibility of learning from responses as an opportunity.

Generate Reference Model

(The technique of asking for a design or code that you will use as a model for your implementation)

Generate Reference Model is the practice that you will probably use most often for critical code. You generate something and then write and test the code yourself.

Generate Reference Model also the default for anything that you can not copy directly into your project – concepts and design ideas.

It might seem like unnecessary work to reimplement something that has been generated for you but it really is valuable work. Reimplementation takes time. It is hard to do it without thinking (thank goodness) so we have many chances to consider the choices that an assistant has made—in detail.

Go Rogue

(Ask for off-the-wall things to gain insight. Ask for all of the responsibilities and then ask for them to be removed and see what is left. Ask for a feature to be removed. This is a catch-all for various investigations. Ask for a merge of several things, a wild change. Any kind of hypothetical)

Reduce Step Size

*(A general strategy to manage cognitive load and avoid fatigue.
Related: ask for finer steps in order to see AI's partitioning of the
work/thing.)*

Ask What's Unnecessary

(When you do this you are asking for a value grading of things that the code does. This query may lead to code simplification. The question can be conditioned on a particular purpose for the code that you specify)

Ask For Reflection

(Ask the tool to reconsider without offering a reason why)

Vary Resolution

(The process of asking within an adjustable constraint, often numerical. Ex. List 7 most important responsibilities.)

Stabilize Iteratively

(Fixed Point Stabilization.. Translate between two forms until stability. Translation example. Mention the message loop criterion)

Generate And Deploy

Generate And Deploy is one of the easiest things you can do in software development so there should be no surprise that it has problems. It is pretty much the form of generative AI that everyone is scared of. You ask for an idea, some code, or a design element and use it in production.

What could go wrong?

A lot. We are confronted squarely with the quality issue. In general, we want make sure that whatever we generate is either low-risk or testable. We know how to test but how do we assess risk?

Risk is a concept with many dimensions. A mistake can cause loss of money, loss of reputation or loss of health or life. There is another dimension that we rarely consider. Fortunately it is one we can often address: discoverability.

Bugs can be easy to discover or hard. If a prominent JPEG image of a company logo is missing, chances are it will be discovered almost immediately. On the other hand, a slight mathematical error in critical computation that manifests only under rare conditions might lie undiscovered for years.

There is a quote commonly attributed to John Backus, a functional programming pioneer: “APL consists mainly of total functions, so most programs will run, but when they do, the errors are usually obvious”

(expand on the discoverability aspect.. monitoring and the categorization of modules)

Completion Check

(Offering something that is incomplete and asking for the completion. This is inherent in anything with context, but it can be an explicit technique. Complete this class, complete this feature set, generate tests for this code.. here are 3 of 10 things, give me the other 7)

Solve Then Ask

Solve Then Ask is solving the problem first and then asking your assistant for a solution. Critically, you don't give your assistant your solution, you simply give it the problem.

This seems redundant, right? Why do it?

I do it because it is a great way to learn. There is nothing that increases our skill more than seeing a different way of solving a problem immediately after we've done it ourselves.

When we see an alternative solution after we've made ours, we wonder why we didn't we didn't take that approach. Sometimes the answer is obvious. AI might give you a solution that you had already considered but discarded for various reasons. Other times, you really learn something. It can be something as small as the use of a library function you were unaware of, or as large as a completely different algorithm or design approach.

We can read about how problems are solved but solving them ourselves invests us at a deeper level. The effort we expended developing our solution is palpable. We get to imagine what it would be like if we were aware of what we have learned.

Solve Then Ask is also a great way to practice humility. When we design, either alone or while working with someone else, it is easy to believe that we have found the *best* way but there are always other solutions. When they are better we should use them unless there is a clear reason not to.

Ask For Review

This technique is simply showing the assistant your solution and asking for an alternative or a critique.

Think of it as review.

Currently, it is the default mode of most IDE-based assistants — they use the state of your project as input, but it is easy to do this with a chat interface also — you just paste in your input or point the assistant to it.

As with other forms of review, frequency and chunk size are important. You can *Ask For Review* in very quick increments as you develop. The benefit of quick increments is that you have less waste if you choose to discard some of your solution based on its recommendations.

Chunk size has a parallel tradeoff. If you present the assistant with a large completed solution you might be overwhelmed by its responses. To be fair, you can modulate then by asking for summaries or only a few at a time but the fact that you are dealing with a much larger unit of work can be cognitively and energetically taxing.

(concentric circles model of attention)

Let It Drive

(Agentic mode + multistep problem solving. Reflexively ask for advice)

Ask For Summary Prompt

(Ask for a prompt that reflects a technique or understanding you have reached so that you can reconstitute that knowledge in a new context. Tradeoff against using generated artifacts as knowledge transfer. The issue of communicating languages, DSLs)

Generate Tests

*(Describe fixity, the lossiness of tests, types of coverage and future directions.. also the energy of work with “struggle” - Math secret).
The technique of asking for cases rather than code.*

Ask For Standards Check

(Simple technique, roles, any oracle)

Ask Again

(We forget about indeterminacy. Asking again gives you a different answer (currently). Do this periodically to get past availability bias and maintain optionality)

Approach Obliquely

(When working with AI you may want to avoid directly asking the question that you want a direct answer to. Work your way toward it by building up background. This avoids answers based in incomplete understanding. This technique is used in communication all the time. Example)

Interrupt Flow

(We can now ask AI to break up large requests into tasks and then execute them. It is often better to ask it to stop and let us examine. We forget that we can do anything during these interruptions. We can make projections, ask for summaries, hypotheticals, alternative takes, etc. This is continuous review, but it works only to the degree that we practice it. Alternative mode: ask for walkthroughs of changes. Ask why. Ask for a tutorial of changes, etc.)

Resources

Wolfram, Stephen - What Is ChatGPT Doing ... and Why Does It Work? This book is a great introduction to LLM internals. It starts with neural networks and moves on to give a sense of how ChatGPT architecture worked around 2023. One particular chapter, *Meaning Space and Semantic Laws of Motion* presents a model of how the human concept of meaning might be encoded.

Gao, et al. - Scaling and evaluating sparse autoencoders (6 Jun 2024) This is a very technical paper but the thing that makes it interesting is summarized in this article from June 6, 2024: *Extracting Concepts from GPT4*. <https://openai.com/index/extracting-concepts-from-gpt-4/> It helped me develop some further intuition about LLM encoding, particularly that there are many more “concepts” in any LLM than there are human words for.

Nelson, et al. Lost in the Middle: How Language Models Use Long Contexts (6 Jul 2023) This paper describes recall behavior of LLMs and how it varies with the position of content in the context window. The parallels to human memory retention are interesting - see *Serial-Position Effect* (https://en.wikipedia.org/wiki/Serial-position_effect). This information was helpful in the development of the *Surfacing Model*.

3Blue1Brown (YouTube channel) - Attention in transformers, visually explained This video series digs deep into the attention mechanism of LLMs. It is a good place to go if you want a deeper view of what happens in the layers during inference.